

# Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías - CUCEI



## Hands-on 1: Análisis Léxico y Autómatas

Compiladores.

Alumna: Marlene Ximena Sandoval García

Código: 219511456

Carrera: Ingeniería en Computación

Sección: D05

Calendario: 2026-A

Profesor: José Antonio Aviña Mendez

## Objetivo.

Desarrollar un programa cuya entrada sea una instrucción del Instruction Set Architecture (ISA) definido en esta práctica y cuya salida muestre su identificación, el objeto instrucción y la secuencia de microoperaciones correspondiente.

El programa deberá:

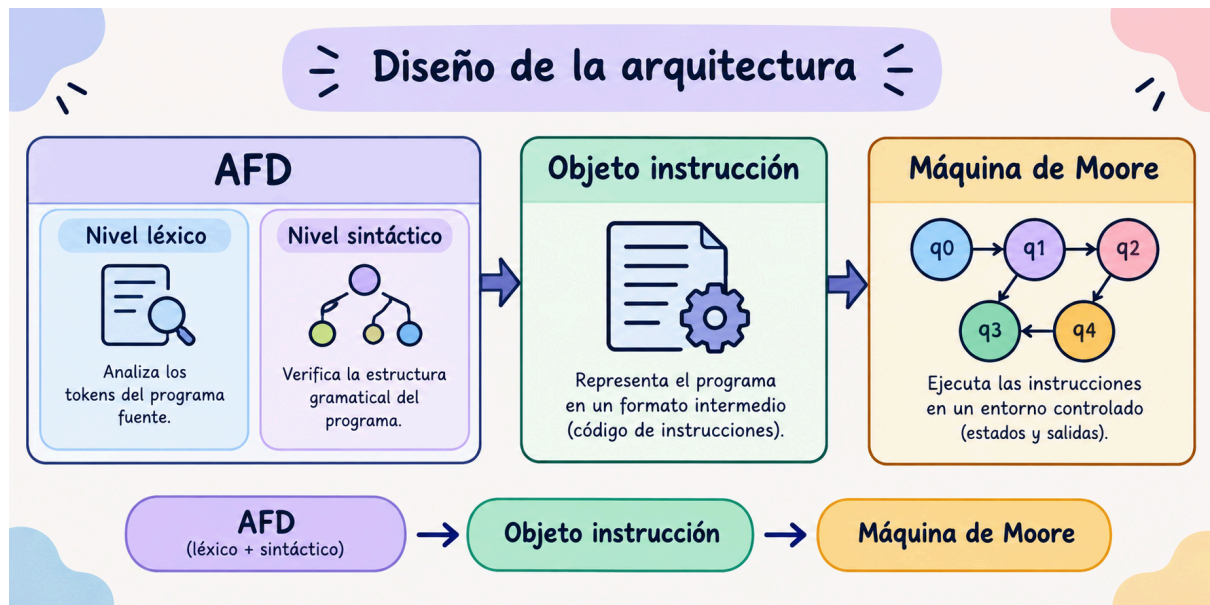
1. Validar la instrucción mediante un autómata finito determinista (AFD).
2. Identificar el mnemónico y sus operandos (argumentos).
3. Construir el objeto instrucción (i.e. representación alterna para una Máquina de Moore).
4. Emplear una máquina de Moore para generar e imprimir las microoperaciones en el orden establecido.

## Arquitectura de referencia.

La arquitectura considera los registros AL y BL (operandos), ACC (acumulador donde se deposita el resultado de la suma), MAR (dirección de memoria en uso) y MBR (dato leído de memoria o por escribir en ella). Además,  $M[d]$  denota el contenido de memoria en la dirección  $d$ .

## Diseño

El diseño se divide en tres componentes que se encadenan: el **AFD** (en dos niveles, *léxico* y *sintáctico*), el **objeto instrucción** y la **máquina de Moore**.



## 1. Estrategia general

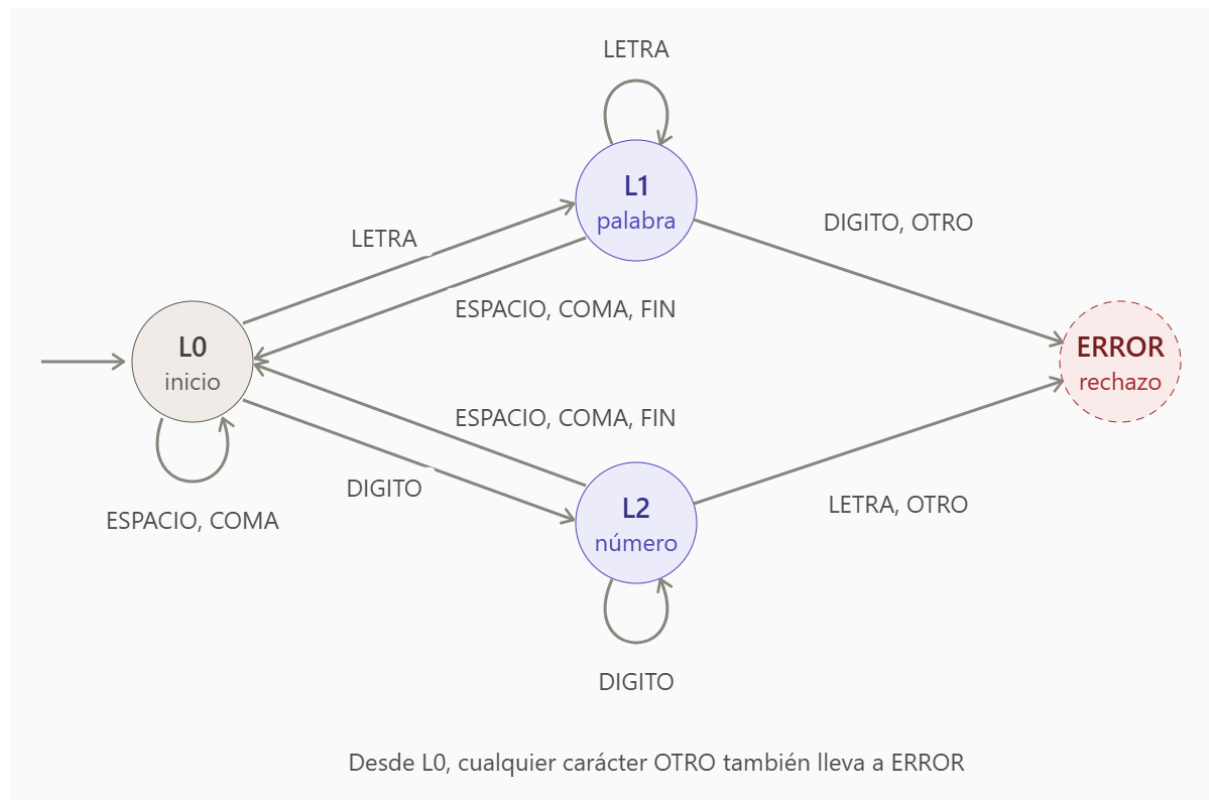
Separé el AFD en dos niveles para que cada autómata tenga una sola responsabilidad:

- **AFD léxico:** recorre la cadena carácter por carácter y produce tokens.
- **AFD sintáctico:** recorre los tokens y comprueba que su orden corresponda a uno de los cuatro formatos del ISA.

Esta separación evita una tabla de transiciones enorme sobre caracteres sueltos y permite dar mensajes de error más precisos.

## 2. AFD léxico

Alfabeto (clases de carácter). **L0** es el estado inicial y **L1** y **L2** acumulan palabras y números. Cualquier carácter inesperado manda al autómata a **ERROR**.

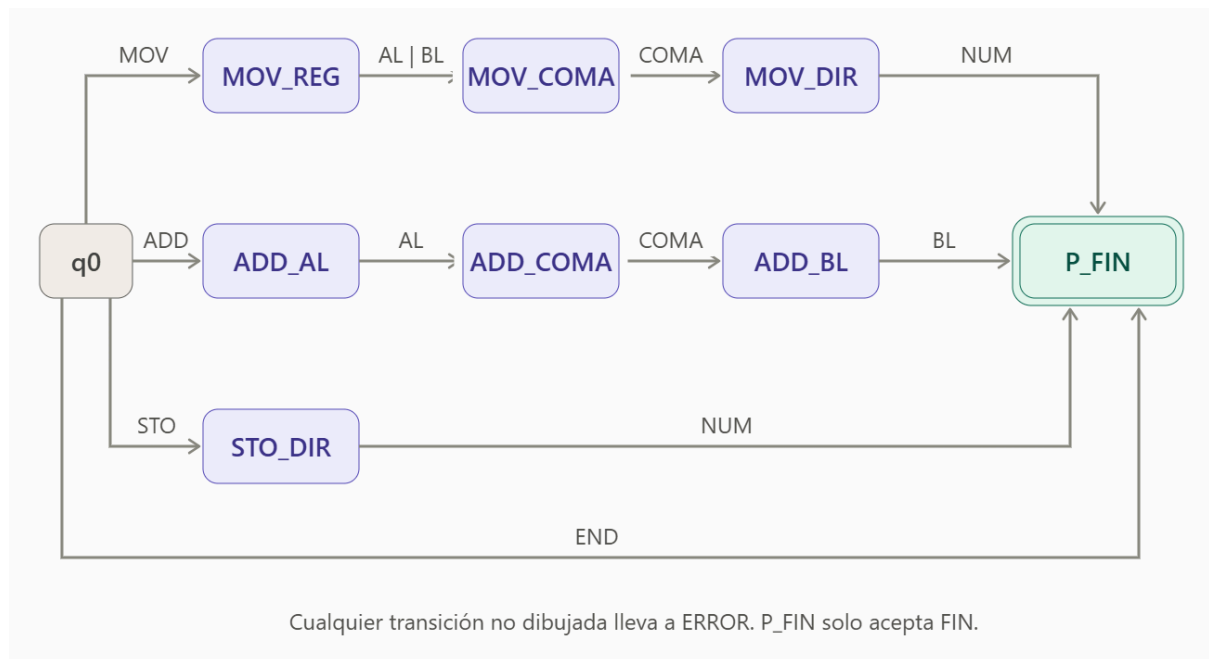


Al cerrar una palabra se clasifica así:

| Lexema                          | Token    |
|---------------------------------|----------|
| ADD, MOV, STO, END              | Mnemonic |
| AL, BL                          | Registro |
| Cualquier otra palabra como: AX | Error    |

### 3. AFD sintáctico

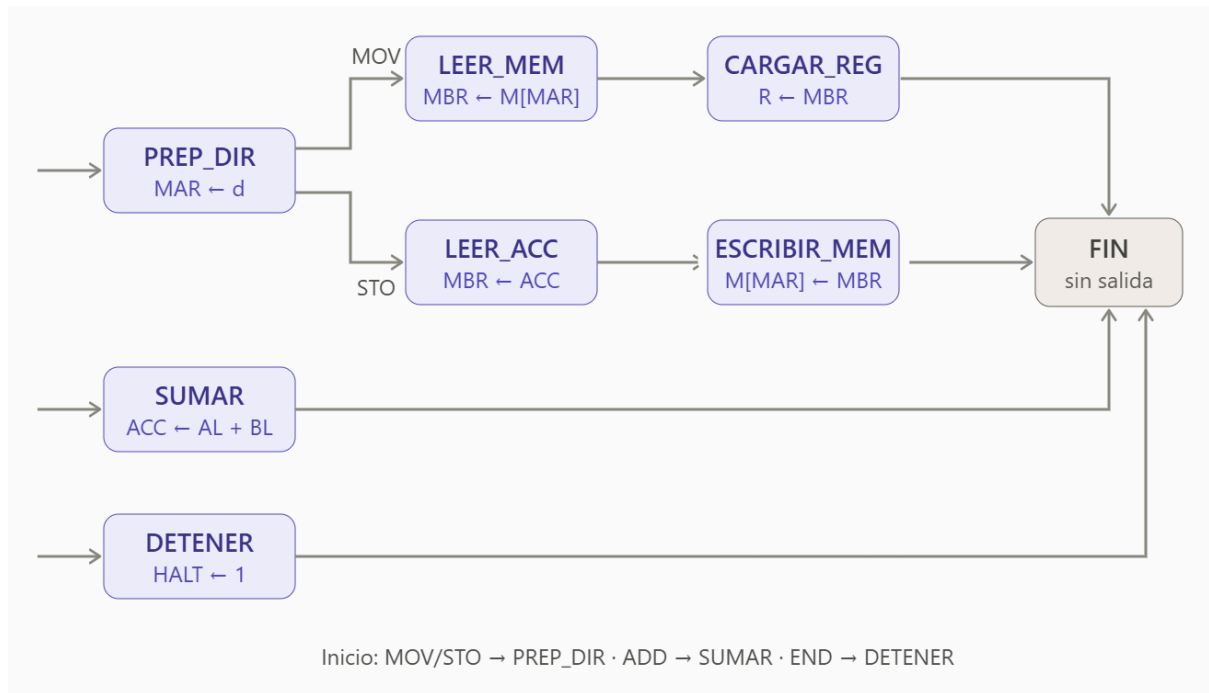
**Alfabeto.** Los tokens que entrega el nivel léxico. **P\_FIN** tiene borde doble porque es el único estado de aceptación. **END** va directo desde **q0**, por eso su flecha da la vuelta por abajo. Si llega cualquier token después de **P\_FIN**, se rechaza por operandos sobrantes.



| Estado   | Significado                                |
|----------|--------------------------------------------|
| q0       | Inicial; espera un mnemónico               |
| MOV_REG  | Leído <b>MOV</b> ; espera registro destino |
| MOV_COMA | Leído el registro; espera coma             |
| MOV_DIR  | Leída la coma; espera dirección            |
| ADD_AL   | Leído <b>ADD</b> ; espera <b>AL</b>        |
| ADD_COMA | Leído <b>AL</b> ; espera coma              |
| ADD_BL   | Leída la coma; espera <b>BL</b>            |
| STO_DIR  | Leído <b>STO</b> ; espera dirección        |
| P_FIN    | Estado de aceptación                       |

### La máquina de Moore.

Máquina de Moore que genera las microoperaciones. Cada estado produce una salida fija (la microoperación escrita dentro de él) y avanza automáticamente al siguiente hasta llegar a **FIN**; la instrucción recibida sólo determina qué ruta se sigue.



## Evidencias.

**instrucción:** **MOV** AL, 6

```

C:\Users\marle\OneDrive\Escri... x + v
Instruccion valida.

TOKENS RECONOCIDOS
MOV("MOV")
REGISTRO("AL")
COMA(",")
NUMERO("6")

COMPONENTES IDENTIFICADOS
Mnemonico: MOV
Registro destino: AL
Direccion de memoria: 6
Direccionamiento: directo

OBJETO INSTRUCCION
{
  operacion: "MOV",
  registros: ["AL"],
  direccion: 6
}

MICROOPERACIONES GENERADAS POR MOORE
1. MAR ← 6
2. MBR ← M[MAR]
3. AL ← MBR
Generacion terminada.

Process returned 0 (0x0)   execution time : 12.123 s
Press any key to continue.
  
```

**instrucción:** MOV BL, 7

```
C:\Users\marle\OneDrive\Escribir archivos>
Instruccion valida.

TOKENS RECONOCIDOS
MOV("MOV")
REGISTRO("BL")
COMA(",")
NUMERO("7")

COMPONENTES IDENTIFICADOS
Mnemónico: MOV
Registro destino: BL
Dirección de memoria: 7
Direccionamiento: directo

OBJETO INSTRUCCION
{
  operacion: "MOV",
  registros: ["BL"],
  direccion: 7
}

MICROOPERACIONES GENERADAS POR MOORE
1. MAR ← 7
2. MBR ← M[MAR]
3. BL ← MBR
Generación terminada.

Process returned 0 (0x0)   execution time : 32.208 s
Press any key to continue.
|
```

**instrucción:** ADD AL, BL

```
C:\Users\marle\OneDrive\Escritorio >
Instruccion: ADD AL,BL
ENTRADA
ADD AL,BL

VALIDACION MEDIANTE AFD
Instruccion valida.

TOKENS RECONOCIDOS
ADD("ADD")
REGISTRO("AL")
COMA(",")
REGISTRO("BL")

COMPONENTES IDENTIFICADOS
Mnemónico: ADD
Registros: AL, BL

OBJETO INSTRUCCION
{
  operacion: "ADD",
  registros: ["AL", "BL"],
  direccion: null
}

MICROOPERACIONES GENERADAS POR MOORE
1. ACC Æ AL + BL
Generacion terminada.

Process returned 0 (0x0)   execution time : 5.884 s
Press any key to continue.
```

**instrucción:** MOV AX,2

```
C:\Users\marle\OneDrive\Escritorio >
Instruccion: MOV AX,2
ENTRADA
MOV AX,2

VALIDACION MEDIANTE AFD
Instruccion invalida: registro invalido 'AX' (solo se admiten AL y BL).
No se construye el objeto instruccion.
No se generan microoperaciones.

Process returned 0 (0x0)   execution time : 9.809 s
Press any key to continue.
```

**instrucción:** MOV AX,2



C:\Users\marle\OneDrive\Escri



Instruccion: ST0

ENTRADA

ST0

VALIDACION MEDIANTE AFD

Instruccion invalida: falta la direccion de memoria en ST0.

No se construye el objeto instruccion.

No se generan microoperaciones.

Process returned 0 (0x0) execution time : 2.602 s

Press any key to continue.